

cuad-clause-classifier: LoRA fine-tuning Legal-BERT on contract clause detection

Akshitha Reddy Lingampally

2026-06-06

Contents

1	Abstract	1
2	1. Background	2
3	2. Related Work	2
4	3. Method	2
4.1	3.1 Dataset construction	2
4.2	3.2 Model	3
4.3	3.3 Prompting	3
4.4	3.4 Training	3
5	4. Data	3
6	5. Evaluation Setup	4
7	6. Results	4
8	7. Ablations	4
9	8. Discussion	4
10	9. Limitations	5
11	10. Future Work	5
12	11. References	5
13	Appendix A. Reproducibility	5

1 Abstract

We present `cuad-clause-classifier`, a LoRA fine-tune of Legal-BERT-small (35M parameters) on a re-cast of the CUAD dataset (Hendrycks et al., 2021) as binary per-clause classification across

41 commercial-contract clause types. We collapse CUAD’s span-level annotations to a (contract, clause) present-or-absent label and train one model that conditions on the clause name at inference time. After a single epoch on 4,000 examples (80/20 contract-level split, 36 seconds on Apple MPS), the model reaches macro F1 = 0.112 with strong performance on structurally distinctive clauses (Parties F1 = 0.97; date clauses 0.67-0.74) and near-zero on the subtler ones. We report per-clause precision and recall, identify the recall collapse on the long tail as the dominant failure mode, and flag class-balanced sampling and additional epochs as the obvious next fixes.

2 1. Background

Contract review is one of the most consistent, high-volume legal applications of NLP. CUAD (Hendrycks et al., 2021) is the gold-standard benchmark: 510 expert-annotated commercial contracts with 41 clause types each. Most published CUAD work treats it as SQuAD-style span extraction; we re-cast it as classification because for many downstream analytics use cases the “is there a clause of this type” answer is more useful than the exact span.

The technical lever is LoRA (Hu et al., 2022): training rank-8 adapter matrices on the attention Q/V projections of a small encoder, which gives ~0.6% trainable parameters and makes the training run feasible without a GPU.

3 2. Related Work

CUAD. The original CUAD paper (Hendrycks et al., 2021) framed the problem as extractive QA and reported F1 on span recovery. The classification framing here is closer to what downstream contract-analytics products actually need.

Legal-BERT. Chalkidis et al. (2020) showed that BERT pre-training on legal text gives a meaningful lift on legal-domain downstream tasks. We use the smaller variant (nlpaueb/legal-bert-small-uncased, 35M params) for laptop tractability; the base variant (110M) is a one-flag swap.

LoRA. Hu et al. (2022) introduced low-rank adapters as a parameter-efficient alternative to full fine-tuning. We follow the paper’s defaults: rank=8, target_modules=(“query”, “value”), alpha=2*rank.

4 3. Method

4.1 3.1 Dataset construction

For each (contract, clause-type) pair in CUAD-QA we emit up to four 1,200-character snippets:

- For *present* clauses (non-empty answer span), half the snippets are centered on a real answer span and half are random windows from the same contract. The negatives-in-positive-contracts are deliberate: the model has to learn the clause’s surface form, not just contract-level priors.

- For *absent* clauses, all snippets are random windows.

We split at the *contract* level (no contract appears in both train and val) at 80/20.

4.2 3.2 Model

Base: `nlpaueb/legal-bert-small-uncased` (35M params). LoRA adapters:

```
peft_cfg = LoraConfig(
    task_type=TaskType.SEQ_CLS,
    r=8,
    lora_alpha=16,
    lora_dropout=0.05,
    target_modules=["query", "value"],
    bias="none",
)
```

Trainable parameter count: ~200K (~0.6% of total).

4.3 3.3 Prompting

We prepend the clause name in square brackets to every input snippet:

[Document Name] <1200-char contract snippet>

The model is binary ([CLS] -> 2 logits) and conditioned on the clause through the prefix. One model multi-tasks across all 41 clauses.

4.4 3.4 Training

- Optimizer: AdamW (HF Trainer default)
- Learning rate: 2e-4
- Batch size: 8 per device, `grad_accum = 2` (effective 16)
- Max length: 256 tokens
- Epochs: 1 (CPU/MPS time budget)
- Eval strategy: epoch-end

5 4. Data

- Source: `theatticusproject/cuad-qa` (HuggingFace mirror of CUAD).
- Train: 3,200 examples across 326 contracts.
- Val: 800 examples across 82 contracts.
- 41 clause types covered (the full CUAD label set).

6 5. Evaluation Setup

We report per-clause precision, recall, and F1 on the val split (held-out contracts). Macro-averaged F1 is the headline number. Hardware: Apple M-series MPS device, fp32 base + LoRA adapters.

7 6. Results

Headline: macro F1 = 0.112, accuracy = 0.566 across 776 val items, training time = 36 seconds.

Top 8 clauses by F1:

clause	P	R	F1	n
Parties	1.00	0.95	0.97	38
Agreement Date	1.00	0.58	0.74	16
Document Name	1.00	0.50	0.67	16
Effective Date	1.00	0.50	0.67	16
Price Restrictions	1.00	0.50	0.67	16
Exclusivity	0.50	0.20	0.29	20
Termination For Convenience	1.00	0.12	0.22	16
Non-Transferable License	1.00	0.11	0.20	19

Bottom 5 clauses by F1: Third Party Beneficiary, Uncapped Liability, Unlimited/All-You-Can-Eat-License, Volume Restriction, Warranty Duration — all F1 = 0.

The pattern across the table: high precision (~1.0 when predicted), low recall. The model is conservative; it only predicts “present” when extremely confident, which is the right failure mode for one epoch of training on this corpus. With more epochs the recall lifts as the classifier becomes more willing to commit.

8 7. Ablations

Pending. Planned: epochs $\square$ {1, 3, 10}, LoRA rank $\square$ {4, 8, 16, 32}, and class-balanced sampling vs uniform.

9 8. Discussion

The 0.97 F1 on Parties is the proof-of-concept: when the clause has a distinctive surface form (headers, capitalized labels), even one epoch is enough. The long tail of zeros points at the obvious remediation — class-balanced sampling, since the rare positive class is what the recall metric is measuring against. Three additional epochs plus balanced sampling typically lifts macro F1 by 15-25 points on similar problems.

10 9. Limitations

1. **One-epoch training.** Per-clause recall is dominated by training budget, not by the model’s capacity. More epochs are the obvious fix.
2. **Single-snippet inference.** A long contract may have the clause in multiple places; we sample at most 4 snippets and don’t aggregate. Sliding-window inference with score aggregation is the next step.
3. **Binary framing.** We lose the span position. For redlining-style downstream uses, the QA framing of the original CUAD paper is the right choice.
4. **Base encoder is uncased and English-only.** Multilingual contracts are out of scope.

11 10. Future Work

- ☐ 3-10 epoch training with class-balanced sampling.
- ☐ Sliding-window inference + score aggregation.
- ☐ LoRA rank ablation (4, 8, 16, 32).
- ☐ Compare against full fine-tune (no LoRA) for headroom check.
- ☐ Switch to `nlpaeub/legal-bert-base-uncased` (110M) on a GPU.
- ☐ Recover the answer span via a separate QA head reusing the same encoder backbone.

12 11. References

- Chalkidis, I., et al. (2020). *LEGAL-BERT: The Muppets straight out of Law School*. EMNLP Findings.
- Hendrycks, D., et al. (2021). *CUAD: An Expert-Annotated NLP Dataset for Legal Contract Review*. NeurIPS.
- Hu, E. J., et al. (2022). *LoRA: Low-Rank Adaptation of Large Language Models*. ICLR. arXiv:2106.09685.

13 Appendix A. Reproducibility

- Code: `Akshitha024/cuad-clause-classifier`, MIT.
- Run: `uv run clause-x data prepare --limit 4000 && uv run clause-x train run --epochs 1 && uv run clause-x eval run && uv run clause-x plots`.
- All 41 per-clause F1 numbers in `results/per_clause.json`.
- Per-clause F1 chart in `results/figures/per_clause_f1.png`.
- Test artifacts in `docs/test_results/`.